CAPSTONE PHASE-1: MENTOR DISCUSSION & WALKTHROUGH GUIDE

Adaptive Human-Review Allocation for Reliable Machine Learning Under Distribution Shift

Student: MCA Candidate (Data Analytics) | **SRN:** PES1PG25CA101 | **Guide:** Prof. Dilip Kumar Maripuri
Department: Department of Computer Applications, PES University, Bangalore | **Course:** UQ25CA741A

1. The 60-Second Elevator Pitch (How to Explain It Informally)

How to Start Your Informal Conversation

"Professor, in high-speed applications like network security, machine learning models process millions of events every day, but human security analysts can only afford to manually inspect a tiny fraction—typically just 1% to 2% due to operational costs and alert fatigue.

The dangerous problem is that when real-world traffic patterns change over time (distribution shift), the model starts making 4 to 5 times more mistakes, but it remains 97% confident in its wrong decisions! So if you only review the predictions where the model looks uncertain, you miss the vast majority of silent, high-confidence errors.

*My project builds an adaptive triage engine that combines **four signals**—prediction uncertainty, how strange the packet is (OOD distance), macroscopic feature drift (PSI), and historical mistake patterns. Under strict human inspection limits (1% to 2%), our multi-factor strategy captures **34.26% of errors** compared to only **26.73%** for confidence alone and just **2.13%** for random checks. I have completely built 13 out of 14 modules across 26 verified Python scripts on 500,000 real flows from the 2026 BigFlow-NIDS benchmark."*

2. The Problem We Are Solving: Why Does ML Fail in Production?

In textbooks, machine learning models are evaluated by taking a dataset, shuffling it randomly, and splitting it 80/20. In the real world, this is a dangerous illusion because time moves forward. An intrusion detection system trained on Monday faces entirely new attacker behavior and shifted network volumes on Friday.

Three critical realities create the operational bottleneck:

- **The Attention Bottleneck:** Human analysts cannot review 100,000 alerts a day. Reviewing costs real time and money. Real SOC teams operate at 1% to 5% review budgets.
- **Silent Overconfidence Failure (Guo et al., ICML 2017 [5]):** When a model encounters shifted data, its accuracy drops, but its probability outputs remain artificially high. In our experiment, errors surged by **4.55x**, but average confidence only dropped from **98.65% to 96.82%**.
- **Random Audit Fails:** Picking alerts randomly captures almost nothing (only 1.17% of errors at a 1% budget in our tests).
- **Single Signal Blindspots:** If you only look at low confidence, you miss confident mistakes. If you only look at Out-of-Distribution (OOD) distance, you flag benign high-speed traffic spikes that aren't actually classification errors.

3. The Dataset & How We Modeled Real Distribution Shift

To make our results rock-solid and impossible to dismiss as synthetic, we used a brand-new 2026 research benchmark:

- **Dataset: BigFlow-NIDS (Data in Brief, Feb 2026) [11]**. It harmonizes four major NetFlow datasets (UNSW-NB15, ToN-IoT, BoT-IoT, CIC-IDS2018) totaling 66.9 million flows with 55 attributes.

- **Our Experimental Sample:** Exactly **500,000 chronological network flows**. We verified that 100% of rows match the raw 14GB source using SHA-256 cryptographic hashes.
- **Chronological Split (Strictly No Future Leakage):** We did NOT use random train-test splitting. We sorted by time:
  1. **Training Split (70% = 350,000 flows):** Baseline attack rate of 6.10% (21,344 attacks). Used to fit models, scalers, and reference covariance matrices.
  2. **Validation Split (15% = 75,000 flows):** Next time window. Attack rate dropped to 0.33% (248 attacks). XGBoost made exactly **753 errors (1.004% error rate)**.
  3. **Shifted Test Split (15% = 75,000 flows):** Future time window. Attack rate surged to 6.30% (4,724 attacks). XGBoost made exactly **3,427 errors (4.569% error rate)**.

Split Partition	Flow Count	Attack Prevalence	XGBoost Error Count	Error Rate	Avg Confidence
Training (70%)	350,000	6.10% (21,344)	Reference Fit	N/A	0.9882
Validation (15%)	75,000	0.33% (248)	753 errors	1.004%	0.9865
Shifted Test (15%)	75,000	6.30% (4,724)	3,427 errors	4.569%	0.9682

4. The Four Reliability Signals: How We Detect Risky Decisions

Instead of relying only on model confidence, our framework extracts **four distinct signals** for every single prediction:

Signal 1: Prediction Uncertainty (s_conf) [1 - max(P(y|x))]

How hesitant is the model? Extracted directly from XGBoost predict_proba. If the model says 51% Benign / 49% Attack, uncertainty is 0.49 (very high). If it says 99.9% Benign, uncertainty is 0.001.

Signal 2: Mahalanobis OOD Distance (s_ood) [Regularized (x-u)^T S^-1 (x-u)]

How unusual is this packet compared to normal training data? It measures multivariate distance taking feature correlations into account, regularized with eps=1e-6 so the covariance matrix is invertible.

Signal 3: Feature Drift Score (s_drift) [Weighted Z-Score x PSI]

Is this packet deviating in features that we know have macroscopically shifted? We calculate Population Stability Index (PSI) across all 49 features. Features with PSI >= 0.10 are flagged, and we score how far each flow deviates along those drifted features.

Signal 4: Historical Mistake Proximity (s_hist) [Decile Error Frequency]

Does this packet resemble past mistakes the model made during training? We discretize 10 key features into 10 deciles and map the empirical error rate of each bin. If a packet falls into bins where the model historically blundered, its risk score increases.

5. What Are the Results? (The Core Slide You Must Defend)

We evaluated 5 strategies across 4 operational human-review budgets (1%, 2%, 5%, 10%) on the 75,000 shifted test flows (where total errors = 3,427):

Review Budget	Uniform Random	Pure Confidence	OOD-Only Filter	Static Multi-Factor	Feedback Adaptive
1.0% 750 reviews	**1.17%** 40 errs	**13.51%** 463 errs	**0.03%** 1 err	**17.51%** 600 errs	**17.80%** **[WIN]** 610 errs
2.0% 1,500 reviews	**2.13%** 73 errs	**26.73%** 916 errs	**0.26%** 9 errs	**33.99%** 1,165 errs	**34.26%** **[WIN]** 1,174 errs

5.0% 3,750 reviews	**5.52%** 189 errs	**65.45% [WIN]** 2,243 errs	**0.35%** 12 errs	**54.57%** 1,870 errs	**45.90%** 1,573 errs
10.0% 7,500 reviews	**9.92%** 340 errs	**84.01% [WIN]** 2,879 errs	**0.35%** 12 errs	**65.42%** 2,242 errs	**52.35%** 1,794 errs

The Three Critical Insights to Tell Your Mentor:

1. **Why Multi-Factor wins at tight budgets (1% and 2%):** When you can only review 750 or 1,500 items, multi-factor triage catches up to **147 more errors at 1%** and **258 more errors at 2%** than confidence alone! It intercepts high-risk drifted packets that confidence completely ignores.
2. **Why Confidence overtakes at broad budgets (5% and 10%):** At larger review budgets, you have enough quota to review the broad decision boundary. Pure confidence is optimal when human resources are abundant.
3. **The OOD Negative Result (Honest Science):** OOD alone caught only 1 error at 1% budget (worse than random!). Why? In network traffic, extreme outliers in feature space are benign high-volume transfers (e.g. large file backups), NOT attacker packets or model errors! Pointing this out proves you didn't fake your results.

6. What Is Actually Implemented? (13 of 14 Modules = 92.9%)

Your mentor may ask: *"Did you just make slides, or is there real code running?"*
Here is the exact proof: We wrote and executed **26 discrete Python scripts** in src/:

Phase / Category	Scripts	Exact Functionality Delivered	Status
1. Ingestion & Split	src/01, 02, 03, 26	Ingests 500K sample, verifies SHA-256 hash against 14GB source, chronological 70/15/15 split, missing value median imputation.	**100% Implemented**
2. Model Training	src/04	Trains Logistic Regression, Random Forest, and XGBoost. Evaluates validation ROC-AUC (0.9228 for XGBoost). Saves .pkl artifacts.	**100% Implemented**
3. Signal Extraction	src/05, 07, 10, 14	Extracts uncertainty, regularized Mahalanobis OOD distance, 49-feature PSI drift, and 10-feature decile mistake risk.	**100% Implemented**
4. Strategy Benchmark	src/06, 08, 09, 13, 20, 24, 25	Implements 5 triage strategies, executes 1%, 2%, 5%, 10% budget allocations on 75,000 shifted flows, outputs comparison CSVs.	**100% Implemented**
5. Interactive Web UI	Phase 2	Lightweight Streamlit/Dash dashboard for live slider control and triage queue inspection.	**Planned (0%)**

7. Slide-by-Slide Informal Guide: What to Say on Each Slide

When presenting the 24-slide deck to your mentor or committee, use this exact informal mental roadmap:

Slide	What to Say Informally in 1-2 Sentences
Slide 1 (Title)	State your name, SRN, Guide (Prof. Dilip Kumar Maripuri), and the title clearly.
Slide 2 (Abstract & RQ)	State the core question: 'Can an adaptive multi-factor mechanism catch more errors than confidence alone under tight human budgets?'
Slide 3 (Problem Scenario)	Explain the 1% human review bottleneck and the silent overconfidence failure.
Slide 4 (Purpose & Scope)	Clarify Phase-1 scope: binary NetFlow classification on 500K sample; Phase-2 scope: cross-domain tabular generalization.
Slide 5 (Background Study)	Briefly mention the 5 existing paradigms: Selective classification, OOD detection, PSI drift, Learning-to-defer, and Human-in-the-loop NIDS.

Slide 6 & 7 (Literature Survey)	Highlight IEEE/Scopus indexing. Mention Cao et al. (ICCV 2023 [1]) and Kim et al. (IEEE TIFS 2024 [6]). Note that PMLR papers have verified IEEE equivalents.
Slide 8 (Comparative Study)	Point to the comparison table showing how our proposed system is the only one combining all 4 signals with explicit budget slicing.
Slide 9 (Users of the System)	Describe the three roles: ML Engineer (builds & monitors), SOC Analyst (reviews queued alerts), Evaluator (audits strategies).
Slide 10 & 11 (Functional Reqs)	Walk through FR1 to FR13 as 100% completed across our 26 scripts; explain FR14 (dashboard) is the Phase-2 target.
Slide 12 (Non-Functional Reqs)	Mention strict reproducibility (seed=42), 1.6-second execution speed, SHA-256 integrity, and modular architecture.
Slide 13 (Tools & Tech)	Highlight Python 3.10, XGBoost, scikit-learn, SciPy, pandas, and ReportLab.
Slide 14 & 15 (Architecture & Flow)	Trace the dataflow: Ingestion -> Preprocess -> XGBoost Inference -> 4-Signal Extraction -> Risk Scoring -> Queue Allocation.
Slide 16 (Database Schema)	Explain the file-based schema: structured CSV tables linked by prediction_id and serialized .pkl models with SHA-256 checks.
Slide 17 (Dataset & Shift)	Explain the BigFlow-NIDS benchmark, chronological splits, and the 4.55x error surge from validation to shifted test.
Slide 18 (Model Performance)	Show XGBoost validation ROC-AUC of 0.9228 and explain the 753 baseline validation errors.
Slide 19 (Module Status)	Demonstrate that 13 out of 14 modules (92.9%) are fully implemented and verified in code.
Slide 20 (Signal & Shift Evidence)	Show the PSI drift measurements (PSI=0.766 for packet length) and explain that the model fails silently with 96.8% confidence.
Slide 21 (Empirical Results)	The climax slide: Show that Multi-Factor wins at 1% and 2% budgets (34.26% vs 26.73%), confidence wins at 5%-10%, and OOD alone fails.
Slide 22 (Limitations & Phase-2)	Be honest: review oracle is simulated, signal weights are heuristic. State Phase-2 plans (SHAP, calibration, UI).
Slide 23 & 24 (References)	Conclude with 10 peer-reviewed papers in IEEE format, plus textbook and software citations.

8. Anticipated Mentor Questions & Bulletproof Answers

Mentor Q1: "Why didn't you just retrain the machine learning model on new data?"

Your Answer: In production, retraining requires newly labeled ground-truth data, which takes days or weeks to curate and audit. Our framework operates *in real time at inference time*. It decides which incoming predictions are too dangerous to trust right now, routing them to human analysts today before any retraining cycle can take place.

Mentor Q2: "Why did OOD detection perform so terribly (0.03% capture)? Did your code have a bug?"

Your Answer: No, sir. We thoroughly debugged this: in high-throughput network data, the highest Mahalanobis distances correspond to extreme feature outliers—like high-bandwidth file downloads or large MTU transfers. These flows are statistically rare, but they are completely benign and correctly classified by XGBoost! In fact, Decile 9 of OOD distance had only 12 errors (0.16% error rate), whereas Decile 7 had 1,799 errors. This proves an important scientific finding: novelty in feature space does not equal classification error in network security.

Mentor Q3: "How did you pick the signal weights for the Multi-Factor strategy? Are they learned?"

Your Answer: In Phase 1, the weights are experimentally defined heuristics (e.g., 50% uncertainty + 50% OOD percentile rank, and regime-based scaling where drift and feedback increase the historical mistake weight). We do NOT claim they are globally optimal. In Phase 2, one of our key objectives is to replace heuristic weights with bilevel optimization or learned routing policies incorporating cost matrices.

Mentor Q4: "Why are precision and F1 low on validation (Precision = 0.04, F1 = 0.05)? Is the model broken?"

Your Answer: The validation split has severe class imbalance: only 248 attacks out of 75,000 flows (0.33% prevalence). When prevalence is that low, any standard threshold produces false alarms that depress raw precision. However, the model's **ROC-AUC is 0.9228**, which confirms high ranking ability. More importantly, our project's goal is not to produce a flawless stand-alone classifier, but to build an efficient *review allocation mechanism* that intercepts the errors the classifier does make.

Mentor Q5: "Are you testing this on live Internet traffic?"

Your Answer: No, sir. This is evaluated on an offline research benchmark—the 2026 BigFlow-NIDS dataset. Evaluating on a reproducible benchmark allows exact ground-truth comparison against all 3,427 errors across 4 fixed budgets. Live network taps are explicitly out-of-scope for Phase 1.

9. Phase-2 Roadmap: What Will You Do Next?

When concluding your conversation with your mentor, clearly present these four Phase-2 objectives:

- **1. Multi-Domain Tabular Generalization:** Expand from network intrusion to medical diagnosis (e.g., MIMIC-IV) and financial fraud (Credit Card Fraud) to show that the multi-factor triage engine is dataset-agnostic.
- **2. Post-Hoc Probability Calibration:** Implement temperature scaling and conformal prediction (Guo et al. [5], Geifman & El-Yaniv [4]) to bound error rates with formal statistical coverage guarantees.
- **3. Instance-Level Explainability (SHAP):** Attach SHAP feature attributions to routed alerts so the human reviewer can see *why* a packet was escalated in under 10 seconds.
- **4. Interactive Web Dashboard (FR14):** Deliver a lightweight Streamlit interface allowing administrators to slide human budget quotas (1% to 10%) and observe triage queues in real time.